VIET NAM NATIONAL UNIVERSITY HO CHI MINH CITY
INTERNATIONAL UNIVERSITY

Fundamental Concepts of Data Security

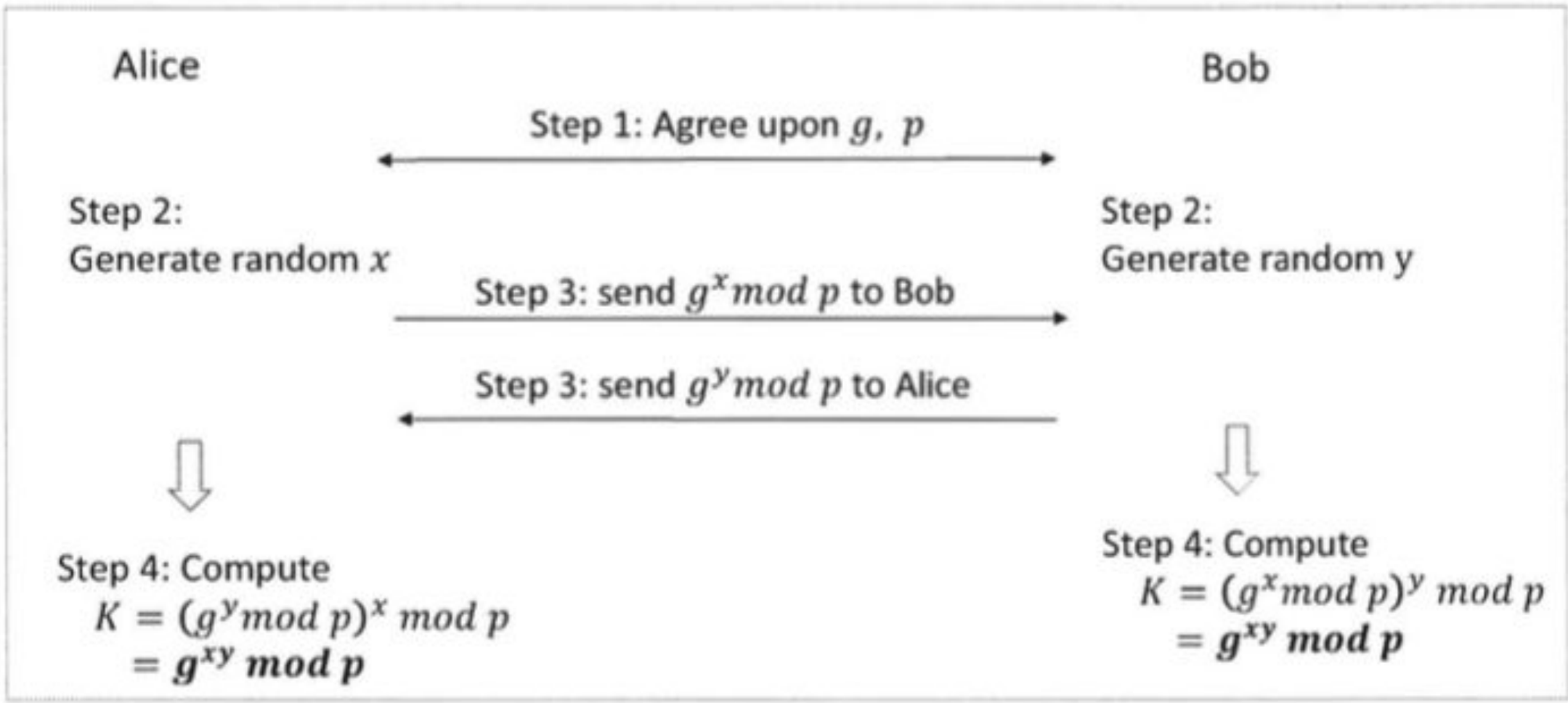
W7 - Public Key Cryptography

Le Hai Duong, PhD. (lhduong@hcmiu.edu.vn)

Challenges of Symmetric Key Encryption

- key exchange problem → how to exchange secret key for two communication parties?
  - face-to-face meeting
  - trusted courier (Grabkey???)

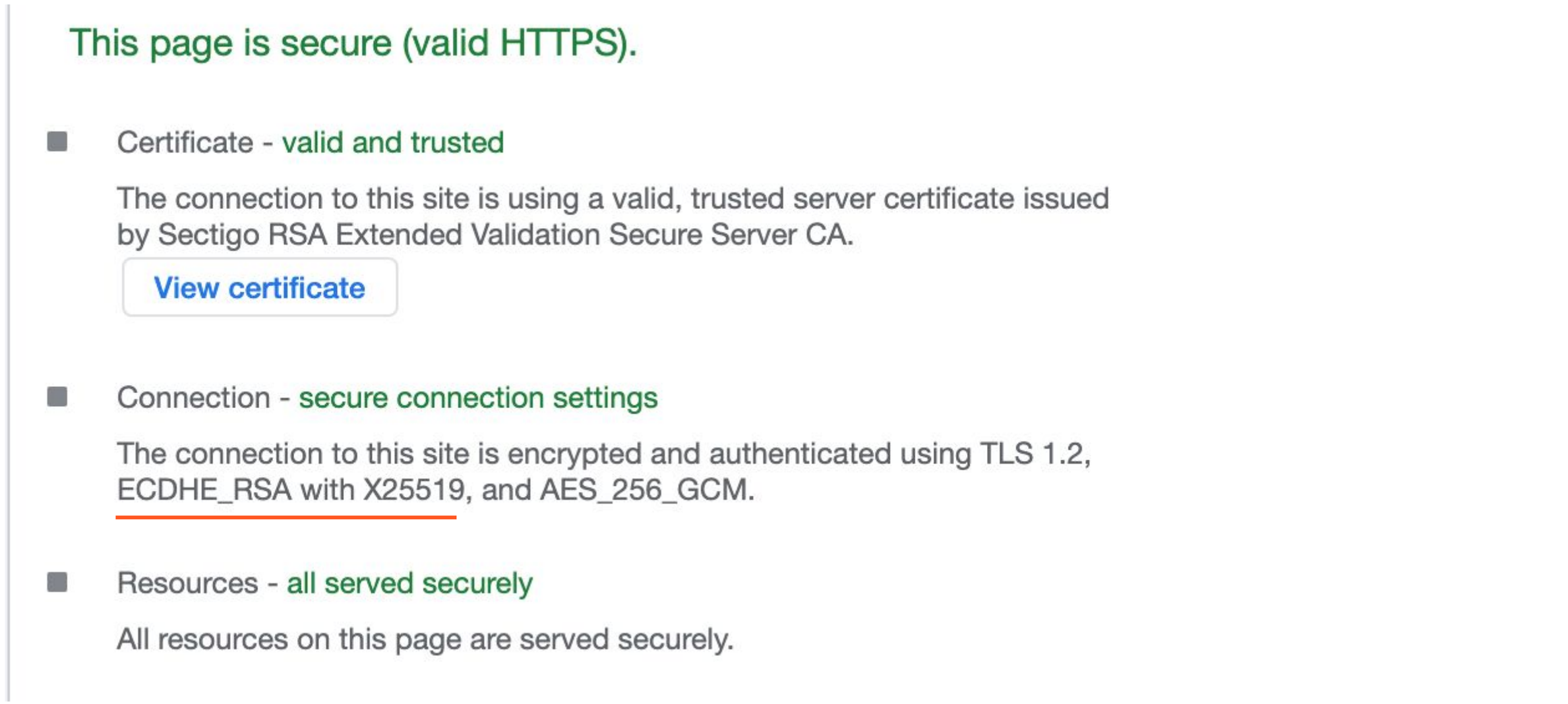
Diffie-Hellman Key Exchange (1976)



p: big prime number (2048-bit number) → finite cyclic group of order p ($\mathbf{Z}_p$)
g: small prime number (e.g., 2 and 3), $g \in Z_p$

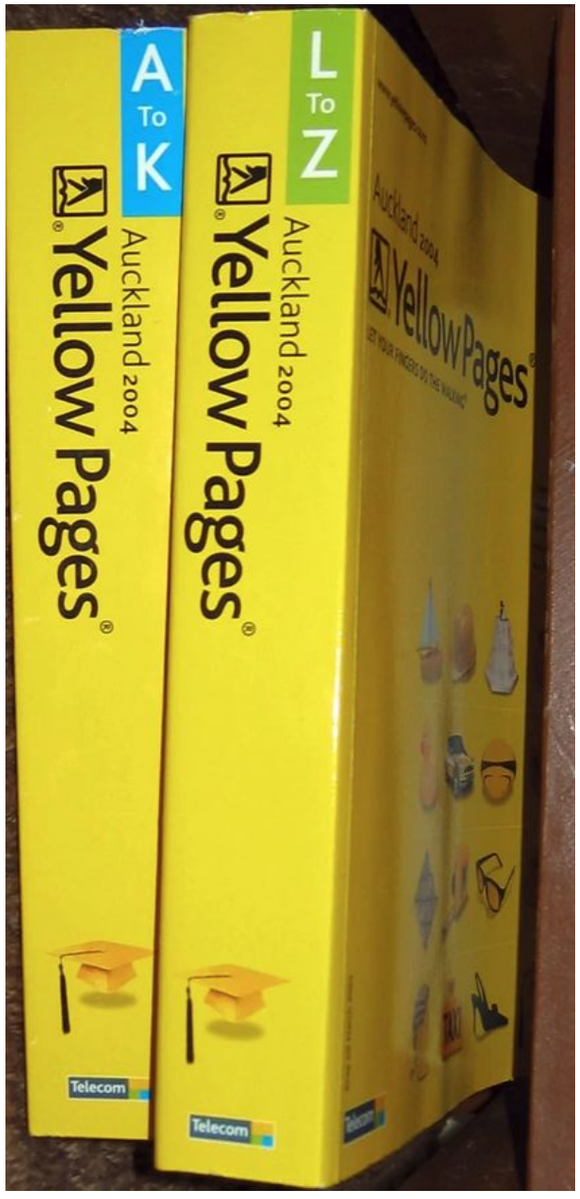
Discrete Logarithm Problem → when x and p are large, finding x from $g^x \bmod p$ will take very long long time, no efficient method is known so far

Elliptic-Curve Diffie-Hellman (ECDH). Instead of performing the computations in a finite field of integers modulo a prime number p, the computations involving x and y in the DH protocol can also be carried out on elliptic curves. This variant is called ECDH (Elliptic-Curve Diffie-Hellman), which is based the Elliptic Curve Cryptograph (ECC). Due to the performance advantage of ECC, ECDH is more often used in practice than the traditional DH protocol.

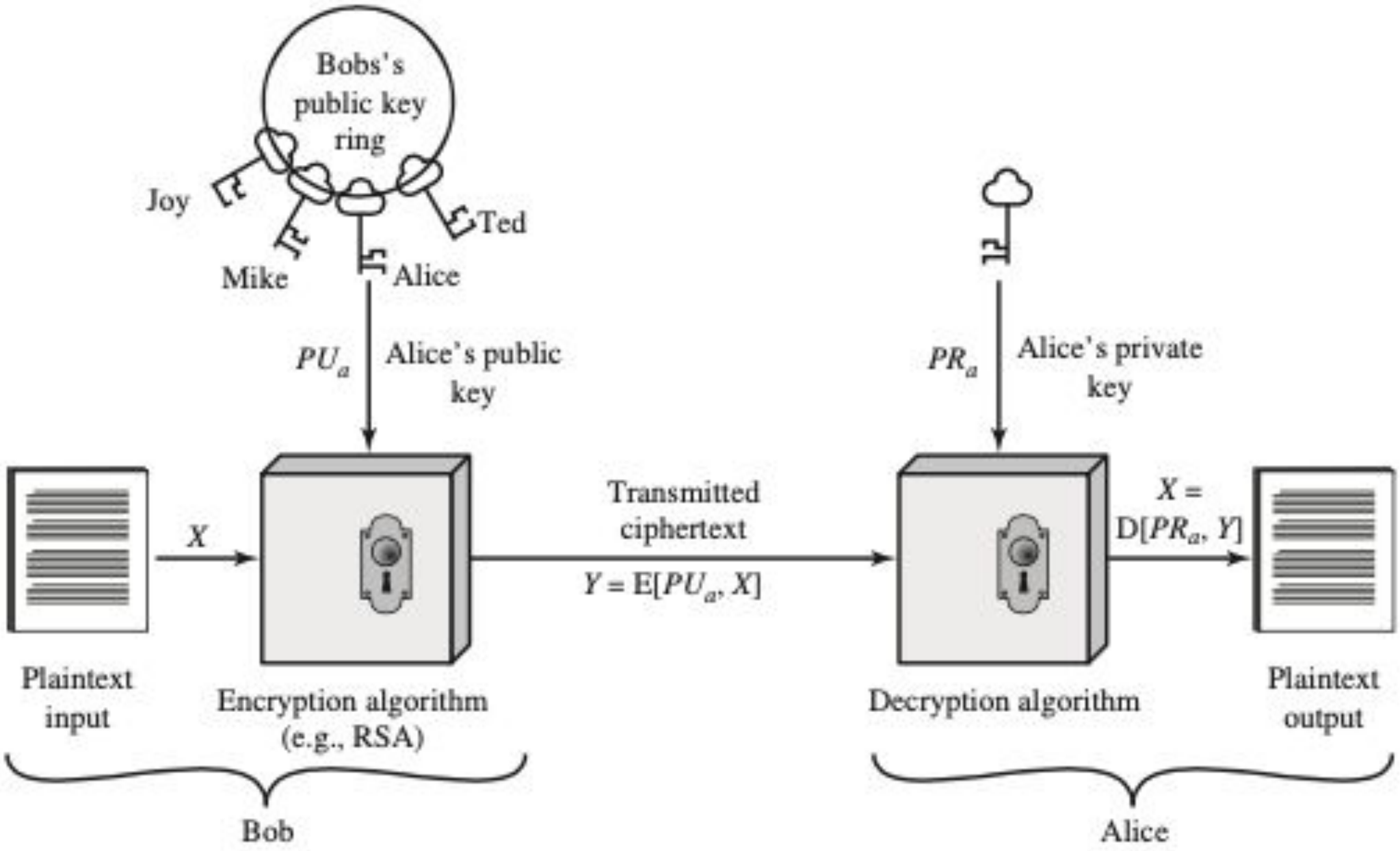


Idea of Public-key encryption





keys could be public like phone numbers in Yellow Pages

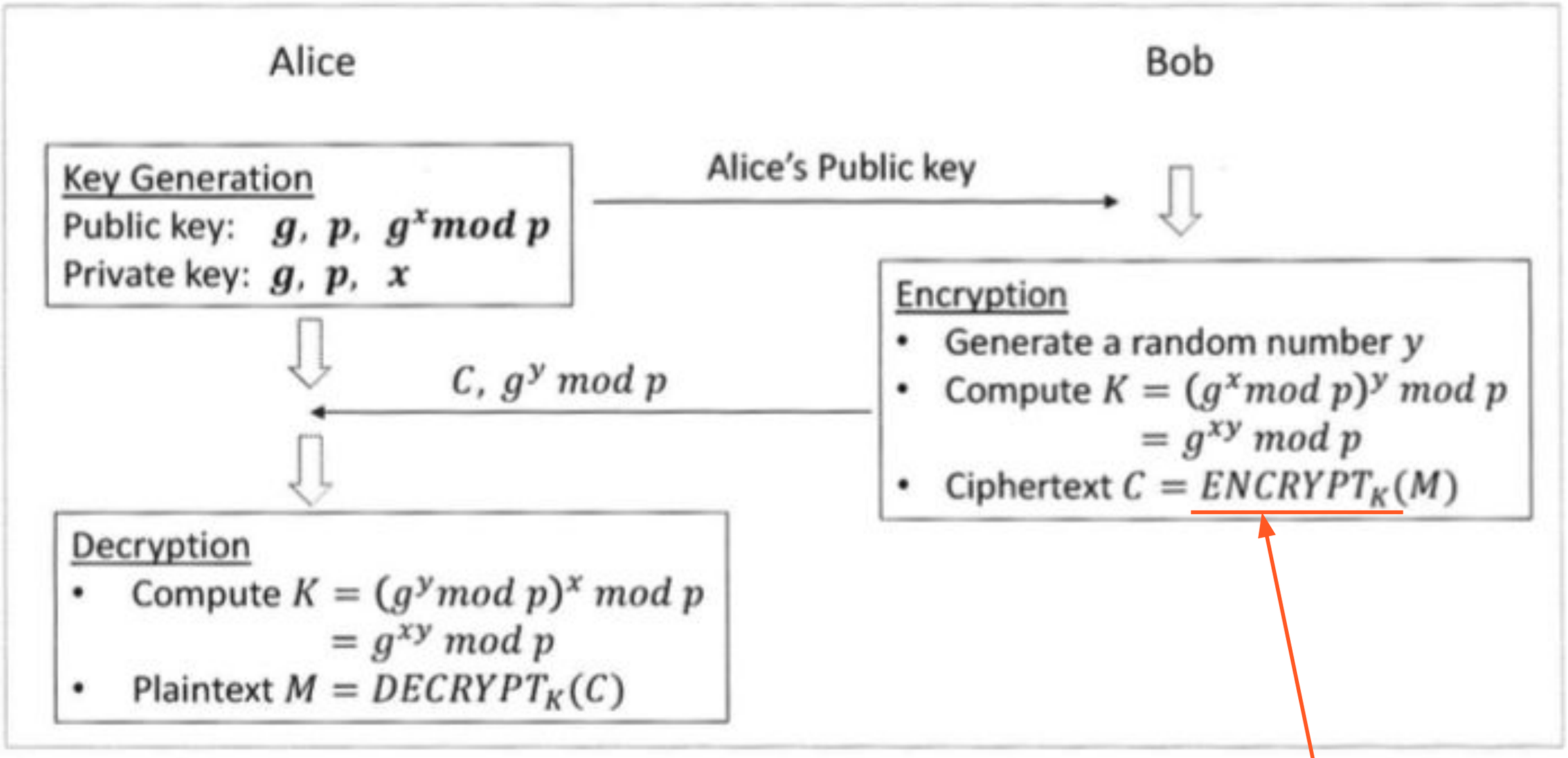


(a) Encryption with public key

Table 2.3 **Applications for Public-Key Cryptosystems**

Algorithm	Digital Signature	Symmetric Key Distribution	Encryption of Secret Keys
RSA	Yes	Yes	Yes
Diffie–Hellman	No	Yes	No
DSS	Yes	No	No
Elliptic Curve	Yes	Yes	Yes

Turning DH key exchange → Public-key encryption



Using the same tweaking to produce public-key encryption:

- ElGamal encryption
- Integrated encryption scheme (IES)

RSA algorithm (1978)
Math background: Modulo operation

$$
\begin{aligned}
(a+b) \mod n &= [(a \mod n) + (b \mod n)] \mod n \\
a * b \mod n &= [(a \mod n) * (b \mod n)] \mod n \\
a^x \mod n &= (a \mod n)^x \mod n
\end{aligned}
$$

$$(14 \, mod \, 3) = 2$$

$$(5 \, mod \, 2) = 3$$

RSA algorithm (1978)
Math background: Euler's Theorem

- Euler's totient function $\phi(n)$ counts the positive integers up to a given integer n that are relatively prime to n
- If n is prime, all integers from 0 to n - 1 are relatively prime to n, therefore $\phi(n) = n - 1$
- If m and n are relatively prime, $\phi(mn) = \phi(m) * \phi(n)$
- Euler's theorem states that

$$a^{\phi(n)} = 1 \ (mod \ n)$$

totient ('təʊʃənt)

n

(Mathematics) a quantity of numbers less than, and sharing no common factors with, a given number

Application of the Euler's Theorem. The theorem may be used to easily reduce large powers modulo n, which is essential for the RSA algorithm. Let us calculate $4^{100003} \mod 33$ by hands. We first calculate $\phi(33)$. After factoring 33 into the product of prime numbers, we get $33 = 3*11$. We calculate $\phi(33)$ in the following:

$$\phi(33) = \phi(3) * \phi(11) = (3-1) * (11-1) = 20$$

Based on Euler's theorem ➔

$$4^{20} \mod 33 = 1$$

and

$$100003 = 5000\phi(33)+3$$

thus, we can compute as follow:

$$\begin{aligned}
&4^{100003} \mod 33 \\
&= 4^{20*5000+3} \mod 33 \\
&= (4^{20})^{5000} * 4^3 \mod 33 \\
&= [(4^{20})^{5000} \mod 33)] * 4^3 \mod 33 \text{ (applying distributive rule)} \\
&= [(\mathbf{4^{20} \mod 33})]^{5000} * 4^3 \mod 33 \text{ (applying distributive rule)} \\
&= 1^{5000} * 64 \mod 33 \text{ (applying Euler's theorem)} \\
&= 31.
\end{aligned}$$

RSA algorithm (1978)
Math background: Extended Euclidean Alg.

- Understand Euclidean alg → how **private keys** are generated in the RSA alg.
- **Euclid's alg.** is an efficient method for computing the greatest common divisor (GCD) of two numbers (the largest number that divides both of them without leaving a remainder).
- **Extended Euclidean alg.** is an extension to Euclidean alg. → not only computes the greatest common divisor of integers a and b, but also find the integers x and y that satisfy the following equation: $ax + by = \gcd(a, b).$

- input: a, b → output: the triple (g, x, y), such that $ax + by = g = gcd(a, b)$

```
def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, x, y = egcd(b % a, a)
        return (g, y - (b // a) * x, x)
```

- RSA alg. uses the extended Euclidean alg. to find the solution to the following equation, where `e` and `n` are components of the **public key** (`e` is relatively prime to $\phi(n)$, i.e., $gcd(e, \phi(n)) = 1$)

$$e * x + \phi(n) * y = gcd(e, \phi(n)) = 1.$$

- the solution `x` will be used as the private key, which is typically called **`d`** ➔

$$e * d + \phi(n) * y = 1.$$

- let both sides modulo to $\phi(n)$

  ➔ given `e` and `n`, find `d` such that

$$e * d \bmod \phi(n) = 1$$

The RSA alg. - Key generation

- Choose two large random prime numbers, p and q.
- Compute $n = pq$. This number is the modulus for the public key and private key. To be secure, n needs to be large.
- Select an integer e, such that $1 < e < \phi(n)$, and e is relatively prime to $\phi(n)$. This number is called public key exponent, and it is made public. This number does not need to be large; in practice, many public keys choose $e = 65537$, which is a prime number. In practice, we fix e first, and then generate p and q. If $\phi(pq)$ and e are not relatively prime (it can happen), we will pick another p and/or q, until the condition is met.
- Find d, such that $ed \mod \phi(n) = 1$. We can use the extended Euclidean algorithm to get d. This number is called private key exponent, and it is kept as a secret.

The tuple (e, n) is the public key.

The RSA alg. - Encryption

Encryption. RSA encryption is quite simple: just treat the plaintext as a number, and then conduct the following modular exponentiation using e and n (assuming that M is less than n; we will discuss the scenario when M is larger than n later):

$$C = M^e \mod n$$

The RSA alg. - Decryption

Decryption. To decrypt a ciphertext C, we conduct the following modular exponentiation using d and n.

$$M = C^d \mod n$$

Let us see whether $C^d \mod n$ can get back M or not, i.e., we need to prove the following:

$$M^{ed} \mod n = M.$$

We know that the public key exponent e and the private key exponent d satisfy a relationship: $ed \mod \phi(n) = 1$. We can get rid of the modulo operator and rewrite the equality as the following:

$$ed = k\phi(n) + 1, \quad \text{where } k \text{ is an integer.}$$

Therefore,

$$
\begin{aligned}
M^{ed} \mod n &= M^{k\phi(n)+1} \mod n \\
&= M^{k\phi(n)} * M \mod n \\
&= (\boldsymbol{M^{\phi(n)} \mod n})^k * M \mod n \quad \text{(applying distributive rule)} \\
&= 1^k * M \mod n \quad \text{(applying Euler's theorem)} \\
&= M
\end{aligned}
$$

We can see from the derivation above that the decryption indeed gets the original message M back.

Exercise: Small number

Let us see how RSA works using actual numbers. We will use small numbers, so we can conduct the calculation using a calculator. First, we choose two prime numbers $p = 13$ and $q = 17$; so we get $n = pq = 221$, and $\phi(n) = (p-1)(q-1) = 192$. Second, we choose $e = 7$ as the public key exponent (7 is relatively prime to $\phi(n)$), and we will use the following equation to find the private key exponent d:

$$ed = 1 \mod \phi(n), \text{ i.e.,}$$
$$7d = 1 \mod 192$$

Solving the above equation (with modulo operator) is equivalent to solving the following equation (without modulo operator).

$$7d + 192y = 1$$

Extended Euclidean Algorithm Calculator
extendedgcd(7,192) - Wolfram|Alpha (wolframalpha.com)

For faster calculation while using much less memory, use the following rules:

$$a^{2k} \mod n = (a^2 \mod n)^k \mod n, \text{ and}$$
$$a^{2k+1} \mod n = (a^2 \mod n)^k * a \mod n.$$

Using the extended Euclidean algorithm, we get $d = 55$ (and $y = -2$). To verify the answer, we calculate $ed = 7 * 55 = 385 = 1 \mod 192$. Now let us encrypt a number $M = 36$:

$$\begin{aligned} M^e \mod n &= 36^7 \mod 221 \\ &= (36^2 \mod 221)^3 * 36 \mod 221 \\ &= 191^3 * 36 \mod 221 \\ &= 179 \mod 221. \end{aligned}$$

$$
\begin{aligned}
C^d \mod n &= 179^{55} \mod 221 \\
&= (179^2 \mod 221)^{27} * 179 \mod 221 \\
&= 217^{27} * 179 \mod 221 \\
&= (217^2 \mod 221)^{13} * 217 * 179 \mod 221 \\
&= 16^{13} * 217 * 179 \mod 221 \\
&= (16^2 \mod 221)^6 * 16 * 217 * 179 \mod 221 \\
&= 35^6 * 16 * 217 * 179 \mod 221 \\
&= (35^2 \mod 221)^3 * 16 * 217 * 179 \mod 221 \\
&= 120^3 * 16 * 217 * 179 \mod 221 \\
&= (120^2 \mod 221) * 120 * 16 * 217 * 179 \mod 221 \\
&= 35 * 120 * 16 * 217 * 179 \mod 221 \\
&= 36 \mod 221
\end{aligned}
$$

[179^55mod 221 - Wolfram|Alpha (wolframalpha.com)]

Recommendations

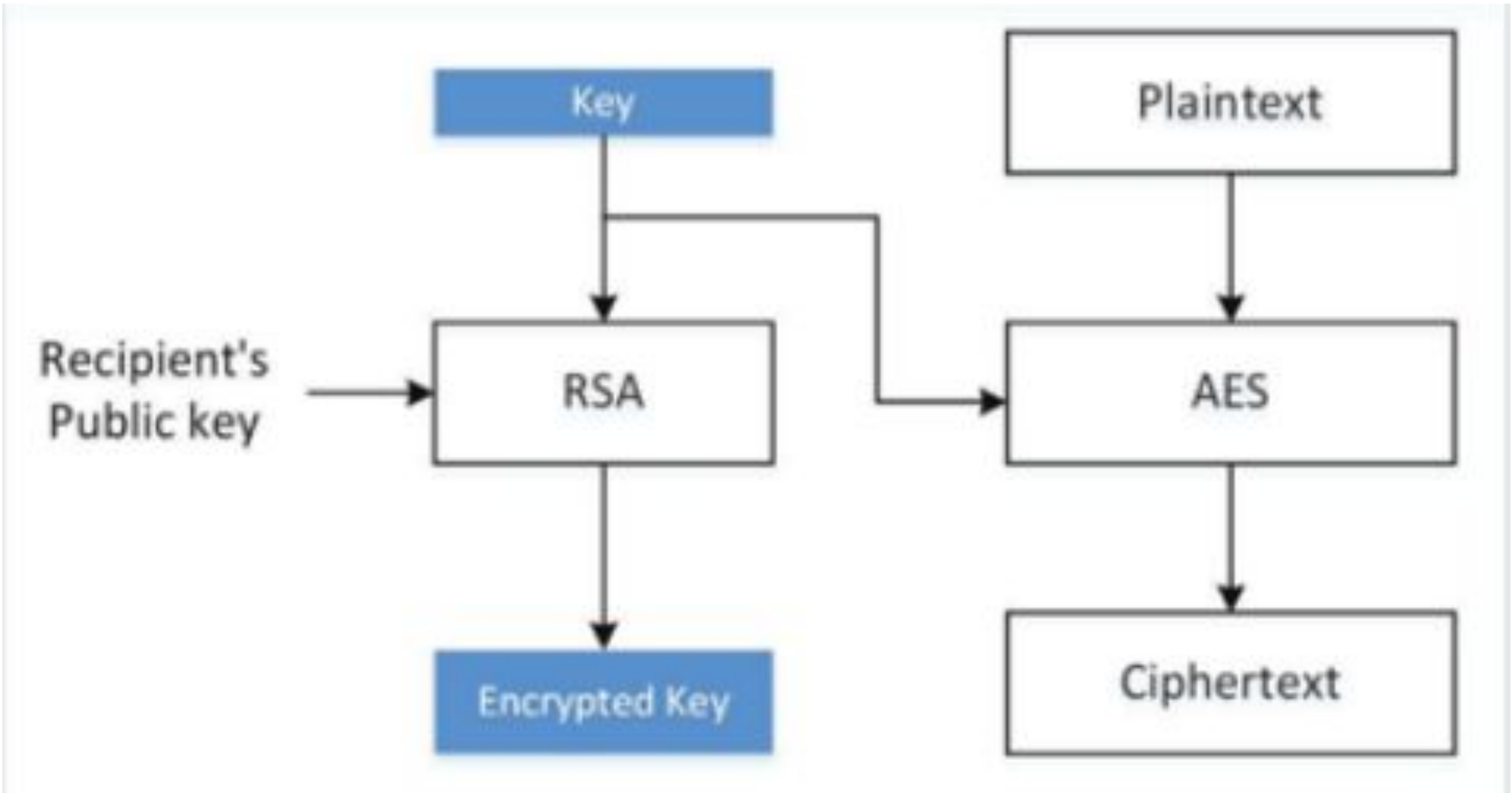
[Key size - Wikipedia]

As of 2016, the NSA's Commercial National Security Algorithm Suite includes:[22]

Algorithm	Usage
RSA 3072-bit or larger	Key establishment, digital signature
Diffie-Hellman (DH) 3072-bit or larger	Key establishment
ECDH with NIST P-384	Key establishment
ECDSA with NIST P-384	Digital signature
SHA-384	Integrity
AES-256	Confidentiality

Hybrid Encryption

- High computation cost of public-key encryption (RSA is 1000 times slower than AES)
- Public key alg. used to exchange a secret session key
- Key (content-encryption key) used to encrypt data using a symmetric-key alg.



Using OpenSSL tools to conduct RSA operations

We will cover:

- Generating RSA keys
- Extracting the public key
- Encryption and Decryption

OpenSSL tools: Generating RSA keys

Example: generate a 1024-bit public/private key pair

private.pem: Base64 encoding of DER generated binary output

```
seed@VM:~/.../rsa$ openssl genrsa -aes128 -out private.pem 1024
Generating RSA private key, 1024 bit long modulus (2 primes)
.....................................+++++
.............................+++++
e is 65537 (0x010001)
Enter pass phrase for private.pem:
Verifying - Enter pass phrase for private.pem:

seed@VM:~/.../rsa$ more private.pem
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: AES-128-CBC,16CC64ADED4603C10DEFA78AF1F2175F

pIbtlC5W0indOG4pZMCTmpq0rkxWhpjVXQjVS0I7nU43ehhovjDC+SKIbGUv9kw8
HvwA2HJ5WPPpowoe4fO3rrmeZu5wvwXeExmHKI4vMSXACK1lqU3z5KlmDRVPK99a
oSKEfurDTRRPfpWo0ijFJH/ubQdOjVYlM6P/xGGKID0PytoI0MV+pGow7eUHr4Tm
K0QitJQhtprZDJlxk8yYRIoVibGOXKn3lIhCdFqNZHsyGbvVunjZVLbfQIqOKRaJ
Tc0k/6iQPBkXv9OftQjwNldr24irdTiCFYXFm+e02+8ww9EWuNeutUesIh9vCGy/
KFLX7qj0vizqFadU0G1j0VfZ8f/wKTDoC0qPyoGU3CY+Drs0OyCG/54xH8DCOZRE
HYnjAK1xvdQRL9du0pEIqDPXXWsSTBwWguQ2HcqDUyRQdm38SCdeSoO8GBm2hjvj
eG28pEdqIHO5UdwLG5qOk529dSaXo1W2gcoDnN53wQRIfzDq4enBfxFSd7Ck8/zy
iO66gZLjK5inDW+um8t5UGFzCYds02QDq20AIcgHWuYGOQ3vM1IWYuybSPRISPhZ
3Ni2wFV2SgJPMYf7bSjTwrCKMUuhtIjxE2xiSnsRRL4tM6qdpozPFiHRu6K9RqFA
3tVkpHGLqCxAI7qZ4yIFNENooGaoGNR2bk2b+bq/m+e8v+s5rqAshkH1AckxNXFZ
7olk724FJ3GyYl0jSJXEi7WBBwTiI2WbBlbtiYUCfHaaOZgCnGs6Tzi+6uhkTSLo
ZrukvtmIi7TtgBzBfOQ1bsYk/xVkyvZxgFwLl3oSyrnmHVQyNLRxjAGrW/SSzWFj
-----END RSA PRIVATE KEY-----
```

Actual content of private.pem

```
seed@VM:~/.../rsa$ openssl rsa -in private.pem -noout -text
Enter pass phrase for private.pem:
RSA Private-Key: (1024 bit, 2 primes)
modulus:
    00:cd:29:c0:8b:40:33:63:af:ac:89:af:0c:78:26:
    1b:de:1d:bc:29:7b:7d:c7:2b:22:3e:8d:3d:2f:b8:
    25:e0:e2:29:15:7b:6c:e7:bb:9a:4f:ce:b7:9b:eb:
    6a:87:87:f9:8c:f7:04:d6:4c:57:d2:dd:38:98:03:
    d9:96:71:aa:dd:a3:b1:8c:7d:cb:c4:89:16:f6:b1:
    68:93:49:86:3e:5c:d6:95:14:a0:f8:2a:49:f6:bb:
    c3:da:bd:72:c5:92:0a:1f:2a:d8:f6:e0:b4:58:96:
    c4:a0:db:fe:f9:60:0e:0f:1d:f1:f4:5d:26:4d:50:
    69:b8:a4:4b:a7:2f:89:c9:6b
publicExponent: 65537 (0x10001)
privateExponent:
    00:c7:73:2f:9c:30:6d:c5:1b:a3:c0:b6:4c:89:01:
    83:56:0b:d2:41:01:ac:e5:53:29:97:65:f7:75:cc:
    b0:06:4f:d7:60:ea:fc:94:cb:ec:d8:69:3e:56:76:
    26:ba:5b:96:5f:54:45:7f:8f:f8:51:26:7f:3e:49:
    7f:de:a1:e2:58:95:e0:5b:a8:00:3c:51:cc:f0:a8:
    94:77:f7:98:6f:fd:eb:0e:d7:82:d1:8f:a2:53:8f:
    ae:bc:54:a4:ef:23:86:7d:5b:56:50:b9:11:7c:06:
    2d:7f:f0:8f:5f:0a:10:28:54:3f:32:97:c2:ba:9d:
    99:9d:03:12:ac:b6:22:c3:01
prime1:
    00:ff:cd:b8:da:81:49:5f:23:35:c7:22:8a:f1:de:
    88:d1:6a:83:5d:3a:22:ea:01:d8:5d:8c:6e:a6:10:
    32:2e:65:eb:2c:dc:cf:91:94:94:e1:5f:ef:52:5e:
    48:d5:75:42:9a:d3:78:f5:18:b1:62:87:70:1d:8a:
    5c:76:f8:bb:a7
prime2:
    00:cd:52:13:a2:eb:f5:5e:a1:d6:d0:cb:5d:cc:77:
    97:1c:38:b7:0e:e4:8b:e3:1c:63:99:7b:64:5c:f0:
    e5:a5:42:18:15:4e:a7:0d:ed:05:0f:5c:2e:c7:ed:
    42:81:26:4a:3e:8e:72:50:4f:1c:3f:d5:f6:1e:ed:
    d8:61:12:2c:9d
```

Extracting the public key

```
seed@VM:~/.../rsa$ openssl rsa -in private.pem -pubout
Enter pass phrase for private.pem:
writing RSA key
-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQDNKcCLQDNjr6yJrwx4JhveHbwp
e33HKyI+jT0vuCXg4ikVe2znu5pPzreb62qHh/mM9wTWTFfS3TiYA9mWcardo7GM
fcvEiRb2sWiTSYY+XNaVFKD4Kkn2u8PavXLFkgofKtj24LRYlsSg2/75YA4PHfH0
XSZNUGm4pEunL4nJawIDAQAB
-----END PUBLIC KEY-----
seed@VM:~/.../rsa$ openssl rsa  -in public.pem -pubin -text -noout
RSA Public-Key: (1024 bit)
Modulus:
    00:cd:29:c0:8b:40:33:63:af:ac:89:af:0c:78:26:
    1b:de:1d:bc:29:7b:7d:c7:2b:22:3e:8d:3d:2f:b8:
    25:e0:e2:29:15:7b:6c:e7:bb:9a:4f:ce:b7:9b:eb:
    6a:87:87:f9:8c:f7:04:d6:4c:57:d2:dd:38:98:03:
    d9:96:71:aa:dd:a3:b1:8c:7d:cb:c4:89:16:f6:b1:
    68:93:49:86:3e:5c:d6:95:14:a0:f8:2a:49:f6:bb:
    c3:da:bd:72:c5:92:0a:1f:2a:d8:f6:e0:b4:58:96:
    c4:a0:db:fe:f9:60:0e:0f:1d:f1:f4:5d:26:4d:50:
    69:b8:a4:4b:a7:2f:89:c9:6b
Exponent: 65537 (0x10001)
```

Content of public.pem

Encryption & decryption

Plaintext

```
seed@VM:~/.../rsa$ echo "The secret is here." > msg.txt
```

Encryption

```
seed@VM:~/.../rsa$ openssl rsautl -encrypt -inkey public.pem -pubin -in
msg.txt -out msg.enc
```

Decryption

```
seed@VM:~/.../rsa$ openssl rsautl -decrypt -inkey private.pem -in msg.enc
Enter pass phrase for private.pem:
The secret is here.
```

Paddings for RSA

- Secret-key encryption uses encryption modes to encrypt plaintext longer than block size.
- RSA used in hybrid approach (Content key length << RSA key length)
- To encrypt:
  - short plaintext: treat it a number, raise it to the power of `e (modulo n)`
  - large plaintext: use hybrid approach (treat the content key as a number and raise it to the power of `e (modulo n)`

Treating plaintext as a number and directly applying RSA is called plain RSA or textbook RSA

Attacks Against Textbook RSA

- RSA is deterministic encryption algorithm
  - same plaintext encrypted using same public key gives same ciphertext
  - secret-key encryption uses randomized IV to have different ciphertext for same plaintext
- For small `e` and `m`
  - if m^e < `modulus n`
  - e-th root of ciphertext gives plaintext
- If same plaintext is encrypted `e` times or more using the same `e` but different `n`, then it is easy to decrypt the original plaintext message via the Chinese remainder theorem

Paddings: PKCS#1 v1.5 and OAEP

- Simple fix to defend against previous attacks is to add randomness to the plaintext before encryption
- Approach is called padding
- Types of padding:
  - PKCS#1 (up to version 1.5): weakness discovered since 1998
  - Optimal Asymmetric Encryption Padding (OAEP): prevents attacks on PKCS
- `rsautl` command provides options for both types of paddings (PKCS#1 v1.5 is default)

PKCS (Public Key Cryptography Standard)- Wikipedia

PKCS Padding

- Plaintext is padded to 128 bytes
- Original plaintext is placed at the end of the block
- Data inside the block (except the first two bytes) are all random numbers
- First byte of the padding is always 00 (so that padded plaintext as integer is less than modulus n)
- Second byte is 00, 01, and 02 (different strings used for padding for different types)

PKCS Padding (Contd.)

```
seed@VM:~/.../rsa$ openssl rsautl -encrypt -inkey public.pem -pubin -in
msg.txt -out msg.enc -pkcs
seed@VM:~/.../rsa$ openssl rsautl -decrypt -inkey private.pem -in msg.enc
-raw > newmsg.txt
Enter pass phrase for private.pem:
seed@VM:~/.../rsa$ xxd newmsg.txt
00000000: 0002 630c 1916 4318 4c54 d98b 6996 c948  ..c...C.LT..i..H
00000010: 0737 2583 46b8 c590 ba61 ed4e 27f8 6150  .7%.F....a.N'.aP
00000020: 10cb ef89 0e68 aa7f 2f3e 96a3 c3fb c17a  .....h../>.....z
00000030: 7214 8fd7 e286 4a94 36f1 b8f0 ffb2 6370  r.....J.6.....cp
00000040: 967c 26f7 ab89 40ae 1432 14ff 5739 86a1  .|&...@..2..W9..
00000050: bdd2 3ed1 5801 d45a 04e5 98d0 561f 43a7  ..>.X..Z....V.C.
00000060: ea19 9398 d209 70e9 5288 9a00 5468 6520  ......p.R...The
00000070: 7365 6372 6574 2069 7320 6865 7265 2e0a  secret is here..
```

OAEP Padding

- Original plaintext is not directly copied into the encryption block
- Plaintext is XORed with a value derived from random padding data

```
seed@VM:~/.../rsa$ openssl rsautl -encrypt -inkey public.pem -pubin -in msg.txt
-out msg.enc -oaep
seed@VM:~/.../rsa$ openssl rsautl -decrypt -inkey private.pem -in msg.enc -raw >
newmsg.txt
Enter pass phrase for private.pem:
seed@VM:~/.../rsa$ xxd newmsg.txt
00000000: 00ae e4e3 a211 de83 a436 a409 26d3 2879  .........6..&.(y
00000010: ba00 3756 28b8 e7d8 3d71 1dfc 405d 19f3  ..7V(...=q..@]..
00000020: 1fa6 18f8 25de ec12 8aac 11f1 1b24 e4f0  ....%........$..
00000030: 0963 2b27 99c6 2ffa ee4b 9dfb 7cf9 3756  .c+'../..K..|.7V
00000040: 645d aca3 7026 0955 b032 f7c5 fb6a d506  d]..p&.U.2...j..
00000050: 93cb a1e4 55de 88d0 f8ff 3355 72a6 e1b1  ....U.....3Ur...
00000060: 16ef fc8c cd37 9a89 5b7c 39ed f87f ead1  .....7..[|9.....
00000070: d404 6747 760b d186 73a8 86ef 9970 a41f  ..gGv...s....p..
```

Digital Signature

- Goal: provide an authenticity proof by signing digital documents
- Diffie-Hellman authors proposed the idea, but no concrete solution
- RSA authors developed the first digital signature algorithm

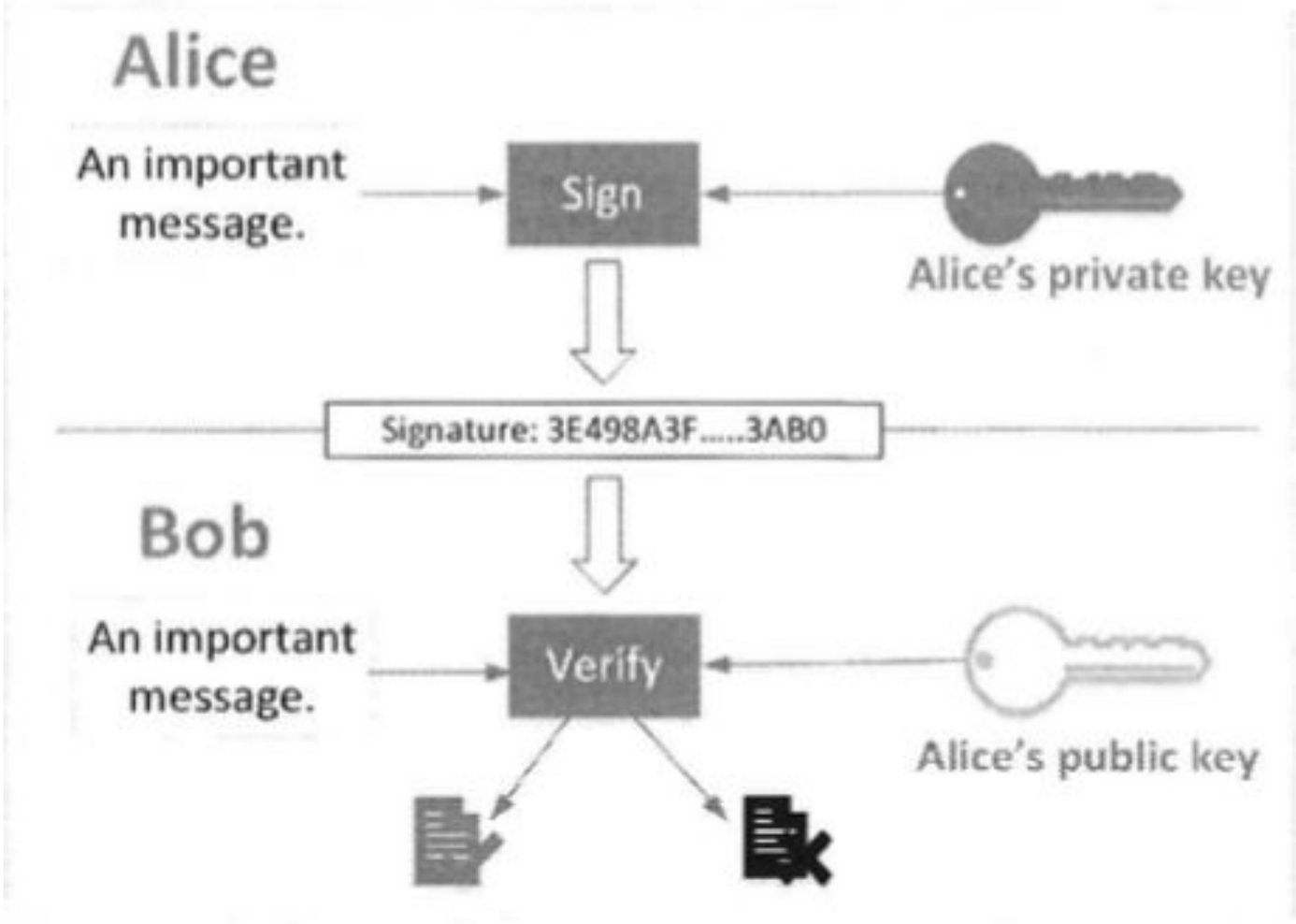


Figure 23.4: Digital signature

Digital Signature using RSA

- Apply private-key operation on m using private key, and get a number `s`, everybody can get the `m` back from `s` using our public key
- For a message m that needs to be signed:

$$\text{Digital signature} = m^d \bmod n$$

- In practice, message may be long resulting in long signature and more computing time
- Instead, we generate a cryptographic hash value from the original message, and only sign the hash

Digital Signature using RSA (Contd.)

Generate message hash

```
seed@VM:~/.../rsa$ openssl sha256 -binary msg.txt > msg.sha256
seed@VM:~/.../rsa$ xxd msg.sha256
00000000: 41f7 830f b7f3 9a37 2830 2722 f9d1 2c71  A......7(0'"..,q
00000010: b224 168b d64e b2fa 010e 573b dbfa 6946  .$...N....W;..iF
```

Digital Signature using RSA (Contd.)

Generate and verify the signature

```
seed@VM:~/.../rsa$ openssl rsautl -sign -inkey private.pem -in msg.sha256
-out msg.sig
Enter pass phrase for private.pem:
[10/16/21]seed@VM:~/.../rsa$ openssl rsautl -verify -inkey public.pem -in
msg.sig -pubin -raw | xxd
00000000: 0001 ffff ffff ffff ffff ffff ffff ffff  ................
00000010: ffff ffff ffff ffff ffff ffff ffff ffff  ................
00000020: ffff ffff ffff ffff ffff ffff ffff ffff  ................
00000030: ffff ffff ffff ffff ffff ffff ffff ffff  ................
00000040: ffff ffff ffff ffff ffff ffff ffff ffff  ................
00000050: ffff ffff ffff ffff ffff ffff ffff ff00  ................
00000060: 41f7 830f b7f3 9a37 2830 2722 f9d1 2c71  A......7(0'"..,q
00000070: b224 168b d64e b2fa 010e 573b dbfa 6946  .$...N....W;..iF
```

Attack Experiment on Digital Signature

- Attackers cannot generate a valid signature from a modified message because they do not know the private key
- If attackers modifies the message, the hash will change and it will not be able to match with the hash produced from the signature verification
- Experiment: modify 1 bit of signature file msg.sig and verify the signature

Attack Experiment on Digital Signature (Contd.)

After applying the RSA public key on the signature, we get a block of data that is significantly different

```
seed@VM:~/.../rsa$ openssl rsautl -verify -inkey public.pem -in
msg.sig.fake -pubin -raw | xxd
00000000: 383e 3e0a 2ffb 67d7 b736 1759 8569 363f  8>>./.g..6.Y.i6?
00000010: 2352 d81a ea6a f439 c400 1b3e c3e0 c649  #R...j.9...>...I
00000020: ad24 65e6 7a83 ef10 b4e9 4f4d f201 9a11  .$e.z.....OM....
00000030: c895 163c 9002 6ecd 17fe dad5 146d 923e  ...<..n......m.>
00000040: c067 90a1 e330 f2b2 8c03 a78d 0607 9c53  .g...0.........S
00000050: 306f e2ff 1565 3042 1108 a7a9 5056 3483  0o...e0B....PV4.
00000060: c278 3eef ea24 4c9c d48d f3a2 caaa 737a  .x>..$L.......sz
00000070: 8528 4a0f 58df 9fa0 ed11 c59e 7714 74d7  .(J.X.......w.t.
```

Programming using Public-Key Cryptography APIs

- Languages, such as Python, Java, and C/C++, have well-developed libraries that implement the low-level cryptographic primitives for public-key operations
- Python:
  - no built-in cryptographic library
  - use Python packages (e.g. PyCryptodome)
- We will cover:
  - Key Generation
  - Encryption and Decryption
  - Digital Signature

Public-Key Cryptography APIs: Key Generation

- Python example (next slide) using Python Crypto APIs to generate a RSA key and save it to a file
- Lines in code:
  - Line (1): generate a 2048-bit RSA key
  - Line (2): export key() API serializes the key using the ASN.1 structure
  - Line (3): extract public-key component

Public-Key Cryptography APIs: Key Generation (Contd.)

```
from Crypto.PublicKey import RSA
key = RSA.generate(2048)
pem = key.export_key(format='PEM', passphrase='dees',)
with open('private.pem', 'wb') as f:
 f.write(pem)
 f.close()

pub = key.publickey()
pub_pem = pub.export_key(format='PEM')
with open('public.pem', 'wb') as f:
   f.write(pub_pem)
   f.close()
```

Public-Key Cryptography APIs: Encryption

- To encrypt a message using public keys, we need to decide what padding scheme
- For better security, it is recommended that OAEP is used
- Lines in code (example on next slide):
  - Line (1): import the public key from the public-key file
  - Line (2): create a cipher object using the public key

Public-Key Cryptography APIs: Encryption (Contd.)

```
from Crypto.Cipher import PKCS1_OAEP
from Crypto.PublicKey import RSA

message = b'A secret message!\n'
key = RSA.importKey(open('public.pem').read())
cipher = PKCS1_OAEP.new(key)
ciphertext = cipher.encrypt(message)
with open('ciphertext.bin', 'wb') as f:
   f.write(ciphertext)
   f.close()
```

Public-Key Cryptography APIs: Decryption

Uses the private key and the decrypt() API

```
from Crypto.Cipher import PKCS1_OAEP
from Crypto.PublicKey import RSA

ciphertext = open('ciphertext.bin', 'rb').read()

prikey_pem = open('private.pem').read()
prikey = RSA.importKey(prikey_pem, passphrase='dees')
cipher = PKCS1_OAEP.new(prikey)
message = cipher.decrypt(ciphertext)
print(message)
```

Public-Key Cryptography APIs: Digital Signature

- In Python code, one can use PyCryptodome library's Crypto.Signature package
- Four supported digital signature algorithms:
  - RSASSA-PKCS1-v1_5
  - RSASSA-PSS
  - DSA
  - RSASSA-PSS
- Show example with RSASSA-PSS

Public-Key Cryptography APIs: Digital Signature using PSS

- Probabilistic Signature Scheme (PSS) is a cryptographic signature scheme designed by Mihir Bellare and Phillip Rogaway
- RSA-PSS is standardized as part of PKCS#1 v2.1
- Sign a message in combination with some random input.
- For same input:
  - two signatures are different
  - both can be used to verify

Public-Key Cryptography APIs: Digital Signature using PSS (Contd.)

- Lines in code example:
  - line (1): create a signature object
  - line (2): generate the signature for the hash of a message

```
from Crypto.Signature import pss
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA


message = b'An important message'
key_pem = open('private.pem').read()
prikey = RSA.importKey(key_pem, passphrase='dees')
h = SHA256.new(message)
signer = pss.new(prikey)
signature = signer.sign(h)
with open('signature.bin', 'wb') as f:
   f.write(signature)
```

Applications

We will cover:

- Authentication
- HTTPS and TLS/SSL
- Chip Technology Used in Credit Cards

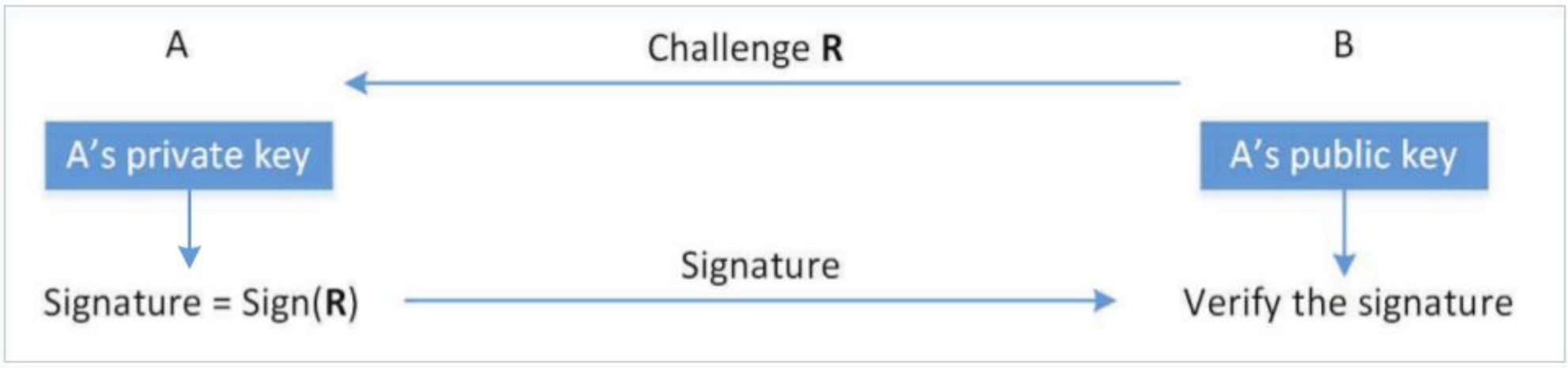
Applications: Authentication

- Typical way to conduct authentication is to use passwords
- Disadvantage:
  - A sends password to B: B can get hacked and A may use same password for multiple accounts
    - cannot be used for many parties to authenticate a single party
- Fundamental problem: password authentication depends on a shared secret

Applications: Authentication (Contd.)

Solution:

- Making the encryption and decryption keys different
- generate the authentication data using one key, and verify the data using a different key



Applications: Authentication (Contd.)

SSH Case Study

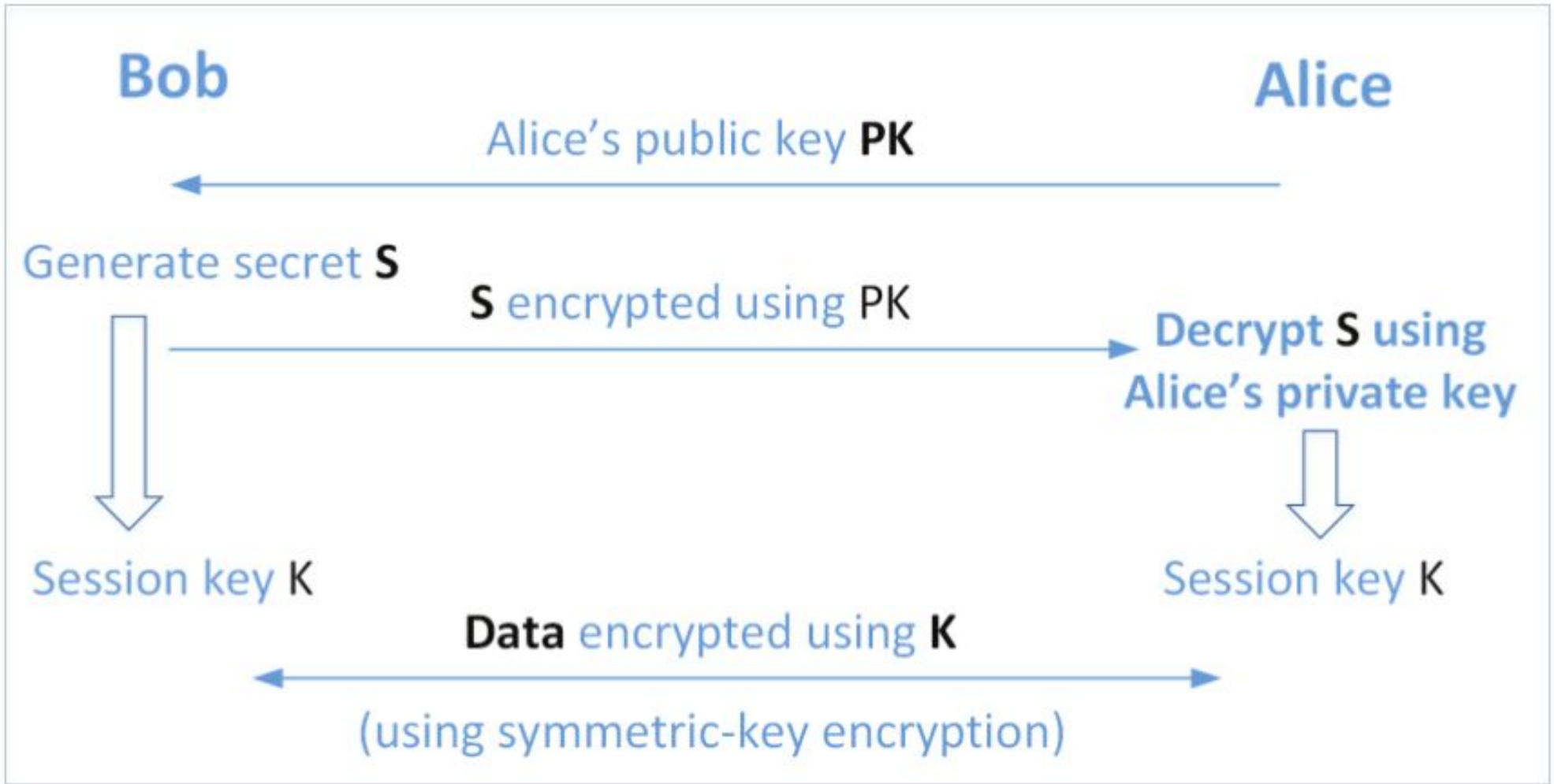
- SSH uses public-key based authentication to authenticate users
- Generate a pair of public and private keys: `ssh-keygen -t rsa`
  - private key: `/home/seed/.ssh/id_rsa`
  - public key: `/home/seed/.ssh/id_rsa.pub`
- For Server:
  - send the public key file to the remote server using a secure channel
  - add public key to the authorization file`~/.ssh/authorized_keys`
  - Server can use key to authenticate clients

https://phoenixnap.com/kb/ssh-with-key

Applications: HTTPS and TLS/SSL

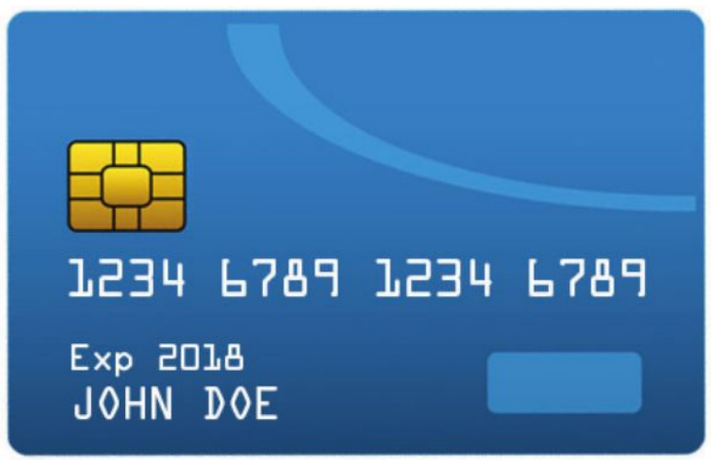
- HTTPS protocol is used to secure web services
- HTTPS is based on the TLS/SSL protocol (uses both public key encryption and signature
  - encryption using secret-key encryption algorithms
  - public key algorithms are mainly used for key exchange

Applications: HTTPS and TLS/SSL (Contd.)



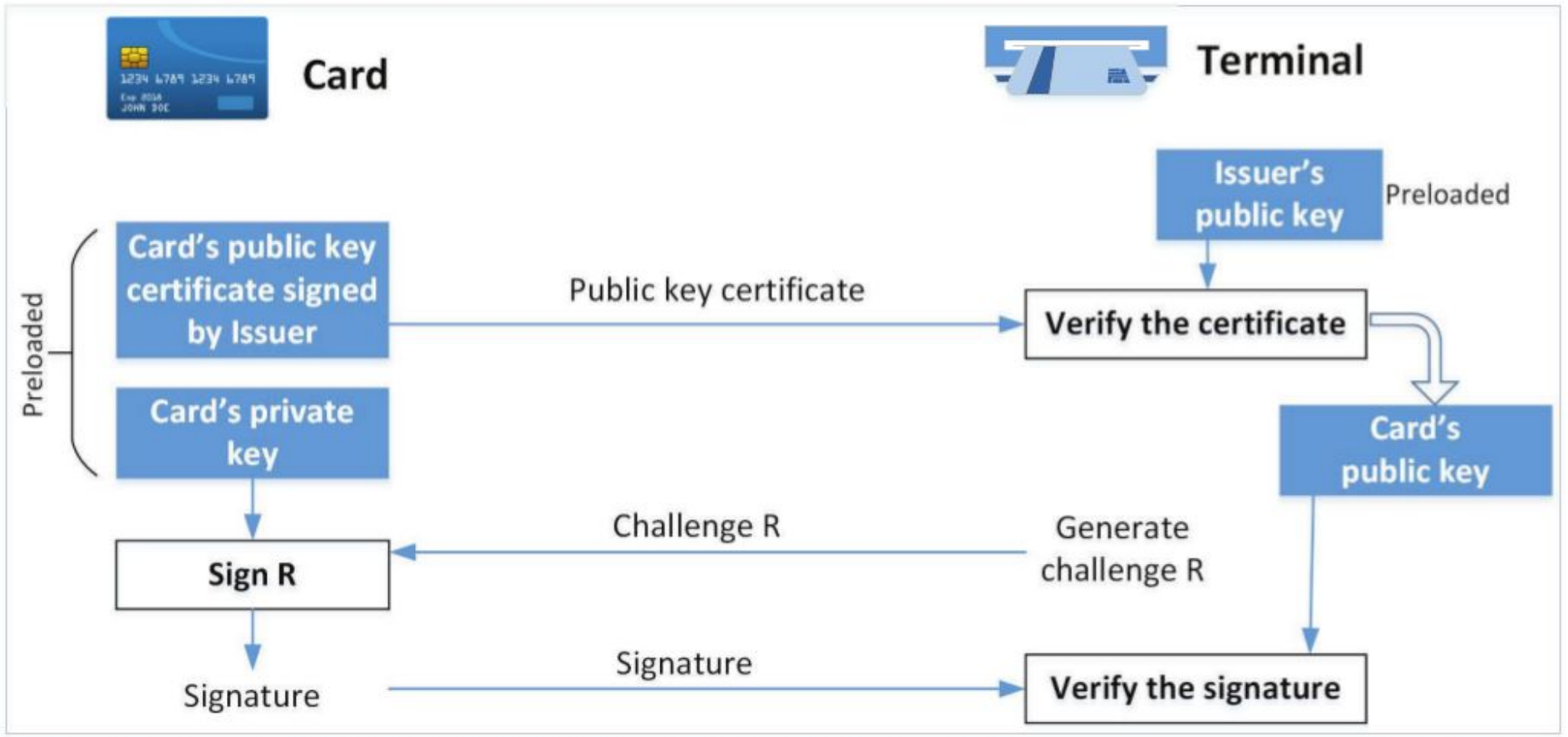
Applications: Credit Card Chip

- Past: cards store card information in magnetic stripe (easy to clone)
- With Chip:
  - chips can conduct computations and store data (not disclosed to outside)
  - EMV standard (Europay, MasterCard, and Visa)
- We will cover how public key technologies are used for:
  - Card authentication
  - Transaction authentication



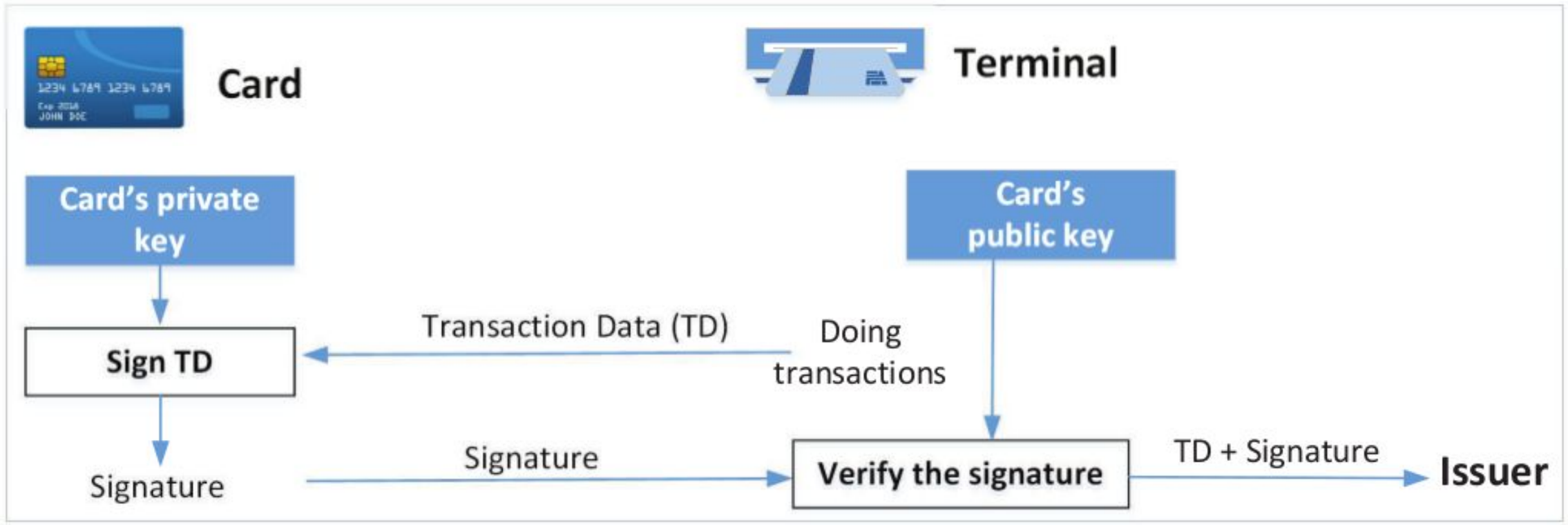
Applications: Credit Card Chip Authentication

- Card contains a unique public and private key pair
  - Private key is protected and will never be disclosed to the outside
  - Public key is digitally signed by the issuer, so its authenticity can be verified by readers



Applications: Credit Card Transaction Authentication

- Issuer needs to know whether the transaction is authentic
- Transaction needs to be signed by the card using its private key
- Verified Signature:
  - To issuers: card owner has approved the transaction
  - To honest vendor: enables the vendor to save the transactions and submit them later



Summary

We covered:

- the basics of public key cryptography
- both theoretical and practical sides of public key cryptography
- RSA algorithm and the Diffie-Hellman Key Exchange
- tools and programming libraries to conduct public-key operations
- how public key is used in real-world applications